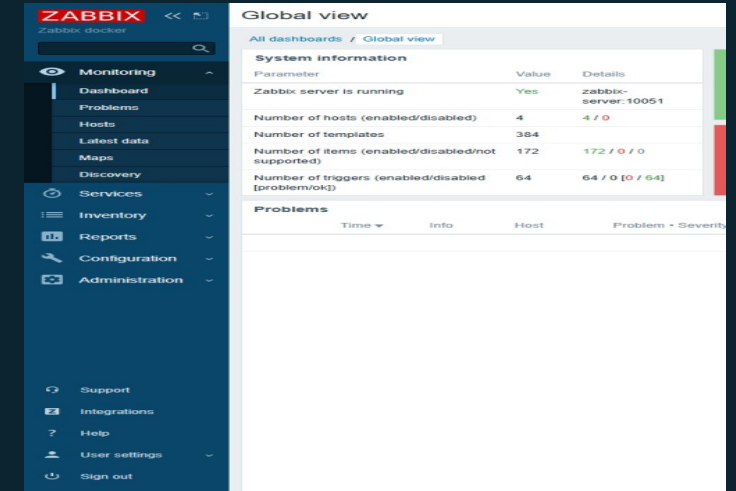


Proyecto 7

Monitoreo de infraestructura con Zabbix



Juan Camilo Ballesteros Sierra

Luis Felipe Murillo Matallana

Juan Sebastian Delgado

Daniela Castro Quinones

Problema

Servicios distribuidos pueden fallar sin aviso.

La revision manual tarda y no deja historial.

Se necesita visibilidad, alertas y evidencia para sustentacion.

Objetivo

Desplegar Zabbix 6.x en Docker Compose.

Monitorear web, base de datos, DNS y FTP.

Validar triggers, dashboards, historicos y alertas por correo.

Agregar portal HTTPS con backend, graficas, SLO y carga Artillery.

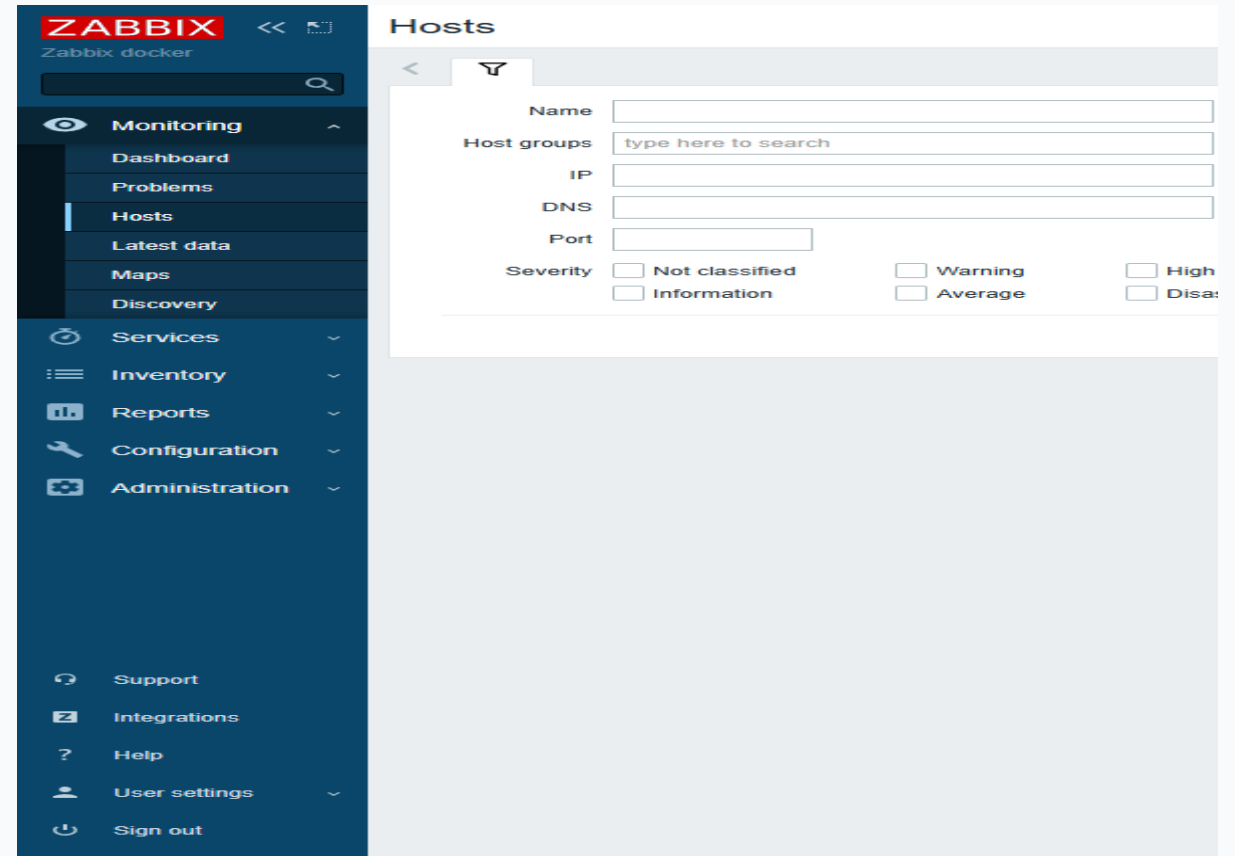
Arquitectura Docker

Zabbix Server + PostgreSQL + Zabbix Web.

MailHog simula SMTP para notificaciones.

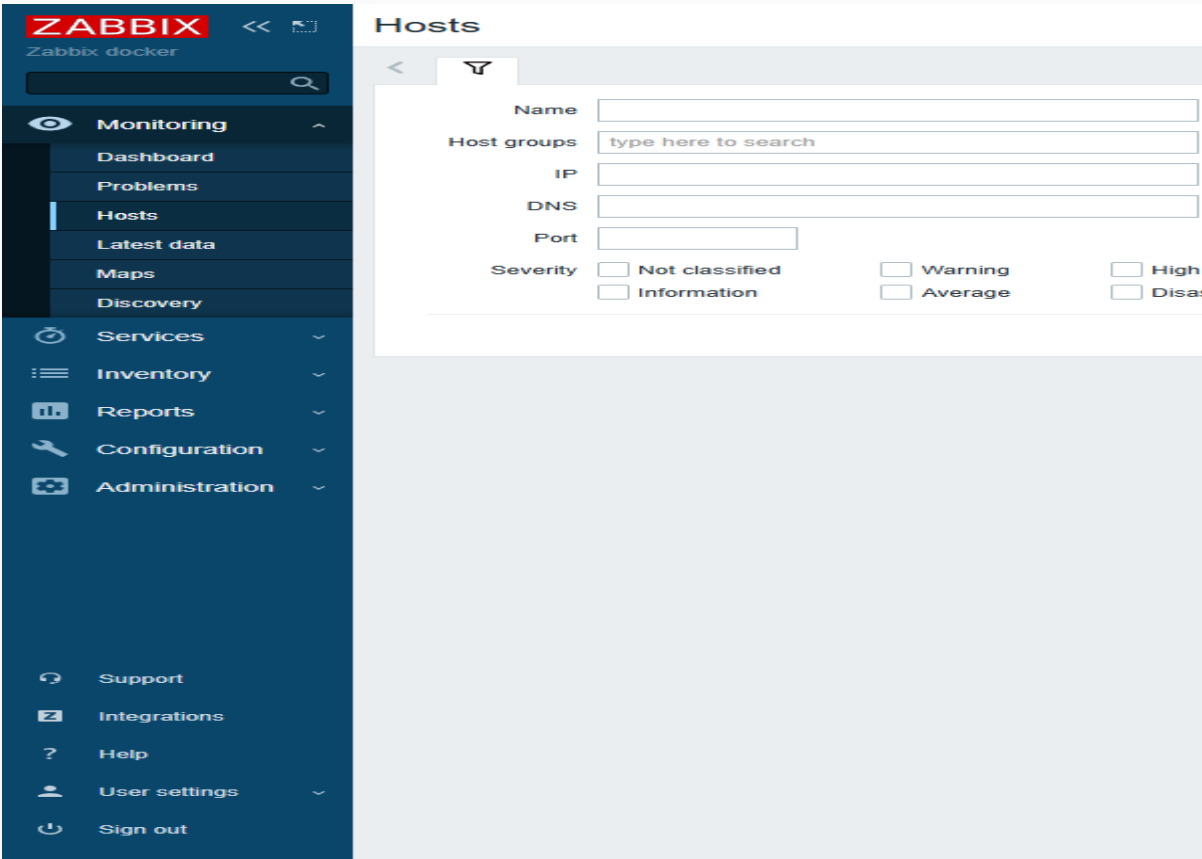
Red interna proyecto7-monitoring para resolución por nombre.

Caddy publica subdominios HTTPS en la VPS.



Inventario monitoreado

- web-host: portal Node.js HTTP.
- db-host: MariaDB puerto 3306.
- dns-host: CoreDNS puerto 53.
- ftp-host: VSFTPD puerto 21.



Implementacion

`docker-compose.yml` define todos los componentes.

Zabbix Server usa imagen personalizada con Dockerfile.

Configuraciones Zabbix se montan como volumen.

`provision_zabbix.py` registra hosts, items, triggers y web escenario por API.

Dashboard y datos

- Latest data muestra disponibilidad y metricas.
- Centro de graficas muestra CPU, memoria, disco, rutas y carga.
- La matriz de cumplimiento /api/compliance cruza requisitos contra evidencia.

ZABBIX

Zabbix docker

Monitoring

Dashboard

Problems

Hosts

Latest data

Maps

Discovery

Services

Inventory

Reports

Configuration

Administration

Support

Integrations

Help

User settings

Sign out

Latest data

Host groups

type here to search

Hosts

type here to search

Name

Subfilter affects only filtered data

HOSTS

db-host 43

dns-host 43

ftp-host 43

web-host 43

TAGS

component 168

TAG VALUES

component: application 4

cpu 68

environment 4

memory 28

os 12

sec

DATA

With data

Without data

Host	Name
☐ dns-host	Disponibilidad DNS dns-service TCP
☐ ftp-host	Disponibilidad FTP ftp-service
☐ web-host	Disponibilidad HTTP web-service
☐ db-host	Disponibilidad MySQL db-service
☐ web-host	Linux: Available memory ?
☐ db-host	Linux: Available memory ?
☐ dns-host	Linux: Available memory ?
☐ ftp-host	Linux: Available memory ?
☐ web-host	Linux: Available memory in % ?

Prueba de caida

Se detiene web-service durante la demostracion.

Zabbix marca el trigger HTTP web-service no responde.

Al restaurar el contenedor, el evento queda resuelto.

ZABBIX Zabbix docker

Monitoring

- Dashboard
- Problems**
- Hosts
- Latest data
- Maps
- Discovery

Services

- Inventory
- Reports
- Configuration
- Administration

Support

- Integrations
- Help
- User settings
- Sign out

Problems

Show: **Recent problems** Problems History

Host groups: type here to search

Hosts: type here to search

Triggers: type here to search

Problem:

Severity: ☐ Not classified ☐ Warning ☐ High ☐ Information ☐ Average ☐ Disaster

Age less than: ☐ 14 days

Time	Severity	Recovery time	Status	Ir
00:06:16	High		PROBLEM	

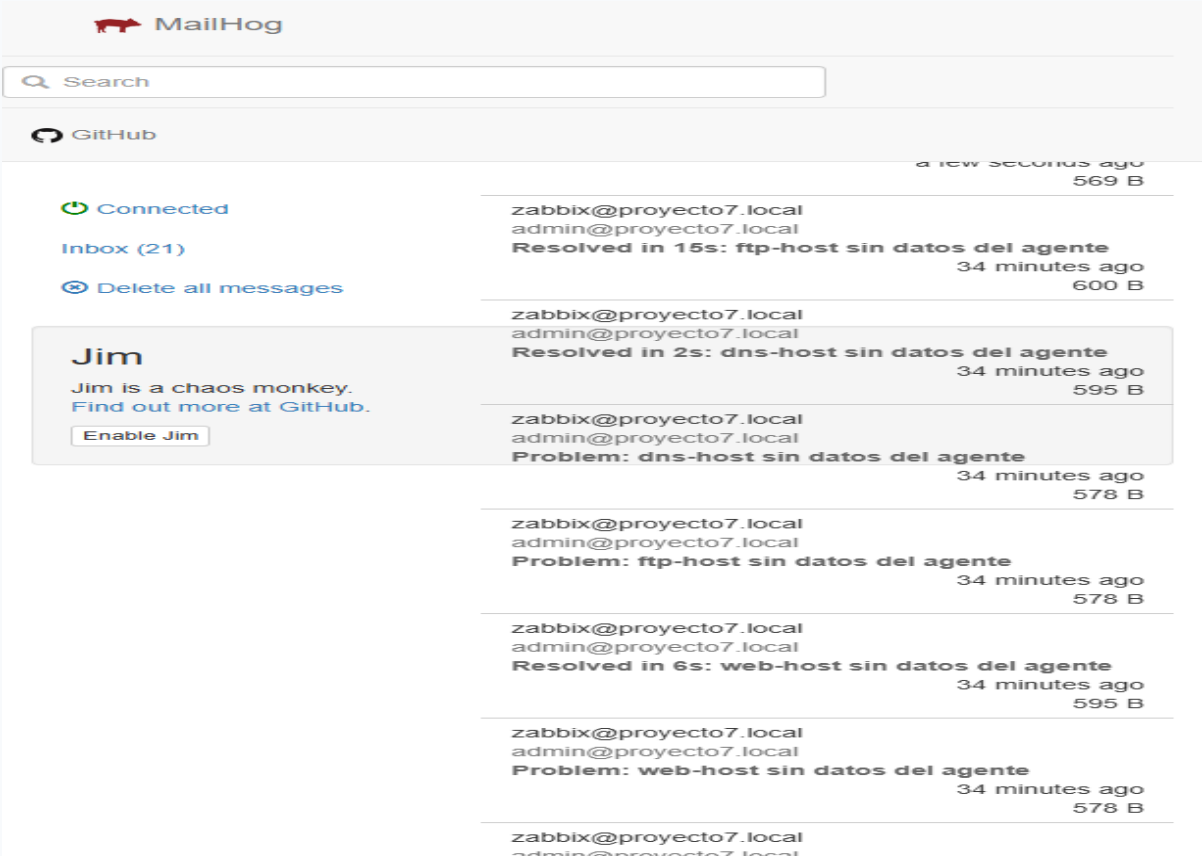
0 selected [Mass update](#)

Alertas con MailHog

MailHog recibe correos de problema y recuperacion.

El portal mailhog-zabbix tiene login propio.

Tambien existe canal SMTP real del dominio para escalamiento.



Pruebas de carga

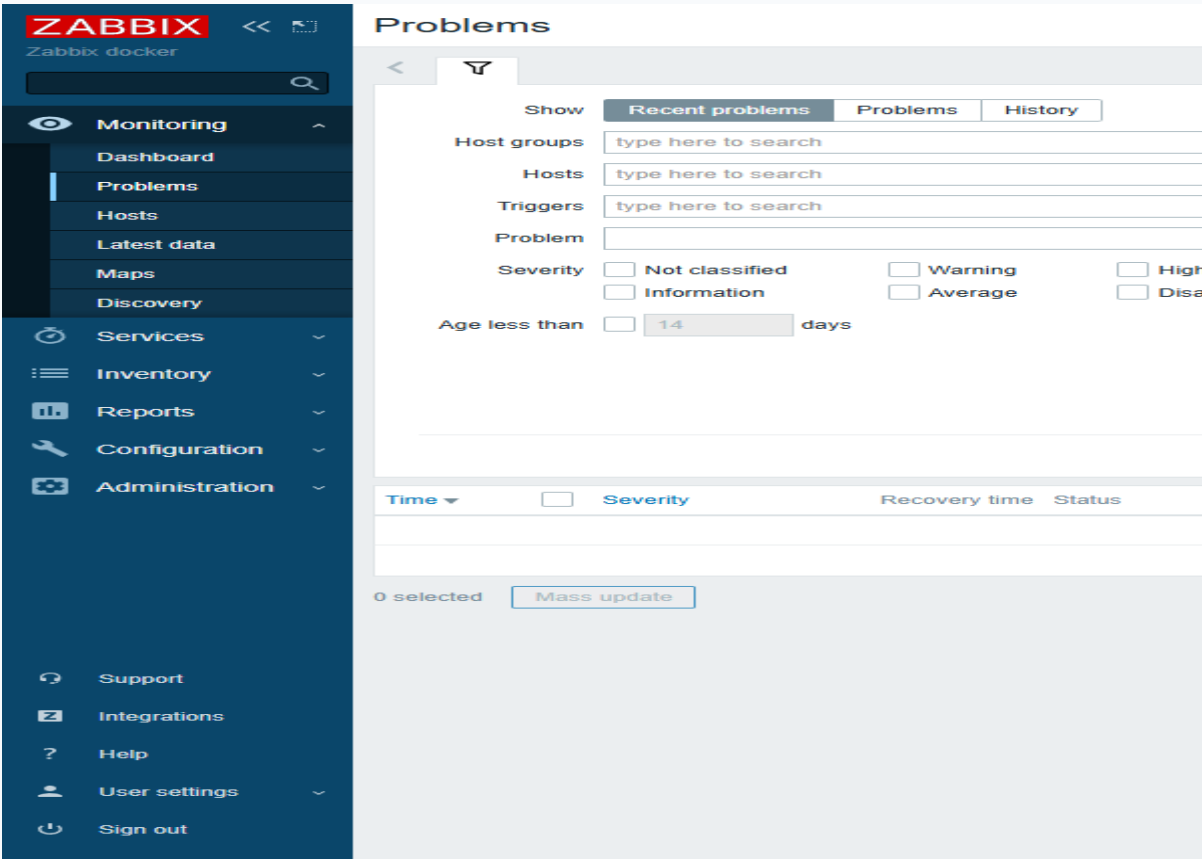
Artillery genera trafico real contra frontend y API.

El backend registra telemetria e incidentes en MariaDB.

Zabbix observa /metrics, DB status y web escenario publico.

Resultados

- Contenedores principales saludables.
- Cuatro servicios con check de disponibilidad activo.
- Alertas generadas y recuperadas durante la prueba.
- Auditoria automatica con 0 fallas esperadas.



Entregables

Informe IEEE: entrega-final/Informe_IEEE_Proyecto7_Zabbix.pdf.

Diapositivas: entrega-final/Presentacion_Proyecto7_Zabbix.pptx.

Repo GitHub con README, Compose, scripts y evidencias.

Conclusiones

Zabbix centraliza observabilidad operativa.

Docker Compose hace el despliegue reproducible.

El portal publico, Artillery y /api/compliance elevan la solucion sobre el minimo.

Demo en vivo

Abrir `web-zabbix.negociocontigo.com`.

Ejecutar: `artillery run tests/artillery-live-demo.yml`.

Simular caída: `docker compose -f docker-compose.vps.yml stop web-service`.

Cerrar con: `bash scripts/audit-project.sh`.